Assignment No. 4

Name : Ashutosh Sanjay More **PRN :** 124B1F036

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git

---

DIJKSTRA'S ALGORITHM

1. Problem Statement

In modern smart city infrastructure, efficient emergency response is critical to saving lives. One of the major challenges is ensuring that ambulances reach hospitals in the shortest possible time, especially during peak traffic conditions.

The city's road network can be represented as a weighted graph, where intersections are represented as vertices and roads are represented as edges. Each edge has a weight indicating the estimated travel time between two intersections based on current traffic conditions.

The objective is to design and implement an intelligent traffic management system using **Dijkstra's Algorithm** to determine the shortest route from an ambulance's current location (source vertex S) to the nearest hospital (destination vertex D).

The system must also support real-time traffic updates by modifying road weights and recalculating the shortest route whenever traffic conditions change.

2. Course Objectives

1. To understand the computational complexity of various algorithms.
2. To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

1. To analyze the asymptotic performance of algorithms.

2. To solve computational problems by applying suitable algorithmic paradigms, including Divide and Conquer and Greedy Methodologies.

4. Theory

4.1 Problem Overview

Dijkstra's Algorithm is a graph-based algorithm used to find the shortest paths from a given source vertex to all other reachable vertices in a weighted graph with non-negative edge weights.

In this assignment, the algorithm is applied to an emergency traffic management system. The vertices represent road intersections, the edges represent roads, and the edge weights represent estimated travel times.

The system identifies the nearest reachable hospital in terms of minimum total travel time rather than geographical distance.

A weighted graph is represented as:

G = (V, E)

Where:

- **V:** Set of vertices representing road intersections.
- **E:** Set of edges representing roads connecting intersections.
- **w(u, v):** Weight of the edge connecting vertices u and v, representing travel time.

The graph is represented using an adjacency list for efficient storage and traversal.

4.2 Greedy Algorithm

Dijkstra's Algorithm follows the **Greedy Method**. At each step, it selects the unprocessed vertex with the smallest tentative distance from the source.

The algorithm then examines the selected vertex's neighboring vertices and updates their distances if a shorter path is found.

The main operations are:

1. Select the vertex with the minimum tentative distance.
2. Examine all its adjacent vertices.
3. Update their distances whenever a shorter path is discovered.
4. Repeat until the destination hospital is reached or no reachable vertices remain.

Since all edge weights must be non-negative, the algorithm can determine the shortest path correctly.

4.3 Steps of the Algorithm

Step 1: Initialization

1. Set the source vertex's distance to 0.
2. Set the distance of every other vertex to infinity.
3. Initialize a parent array to reconstruct the shortest path.
4. Insert the source vertex into a min-priority queue.

Step 2: Select the Minimum-Distance Vertex

Remove the vertex with the smallest tentative distance from the priority queue.

If the extracted distance is greater than the currently recorded distance, discard that entry because a better route has already been discovered.

Step 3: Relaxation of Edges

For every neighboring vertex v of the selected vertex u, calculate the alternative distance:

Alternative Distance = dist[u] + w(u, v)

If this alternative distance is smaller than the current distance to v, update its distance and parent, and insert the updated entry into the priority queue.

Step 4: Check for a Hospital

If the selected vertex is a hospital, the shortest route to the nearest reachable hospital has been found. The algorithm reconstructs the path using the parent array and terminates.

If no hospital is reachable, the algorithm reports that no valid route exists.

Step 5: Real-Time Traffic Update

When traffic conditions change, the system updates the weight of the affected road and executes Dijkstra's Algorithm again.

The new shortest route is then calculated using the updated travel times.

4.4 Path Reconstruction

A parent array stores the predecessor of each vertex along the currently known shortest path.

Whenever a shorter route to a vertex is discovered, its parent is updated.

After reaching the destination hospital, the system traces the parent array backward from the destination to the source. The resulting sequence is reversed to obtain the shortest path from the ambulance's location to the hospital.

4.5 Working of the Greedy Algorithm

Greedy-Choice Property

At every iteration, Dijkstra's Algorithm selects the unprocessed vertex with the smallest tentative distance.

Because all edge weights are non-negative, no alternative route through an unprocessed vertex can produce a shorter distance to the selected vertex. Therefore, its shortest distance is finalized.

Optimal Substructure

A shortest path consists of shortest paths to its intermediate vertices. If a shorter route to an intermediate vertex existed, replacing the corresponding section would produce a shorter overall route.

This property allows the algorithm to construct optimal routes by repeatedly relaxing edges.

4.6 Real-Time Traffic Management

In a smart city, road travel times can change because of congestion, accidents, road closures, or other traffic conditions.

The proposed system supports these changes through the following process:

1. Receive updated travel-time information for a road.
2. Update the corresponding edge weight in the graph.
3. Execute Dijkstra's Algorithm using the ambulance's current location.
4. Identify the nearest reachable hospital based on the updated travel times.
5. Display the revised shortest path and total estimated travel time.

This implementation recalculates the route whenever an edge weight changes. It does not implement a fully incremental shortest-path algorithm; instead, it recomputes the result using the updated graph.

5. Algorithm

Algorithm: Dijkstra's Algorithm for the Nearest Hospital

Input:

- Number of vertices V.
- Number of edges E.
- Weighted edges representing roads.
- Set of hospital vertices.
- Source vertex representing the ambulance's location.

Output:

- Minimum travel time to the nearest reachable hospital.
- Shortest path from the source to that hospital.
- Updated shortest path after a road weight changes.

Procedure:

1. Construct an adjacency list representing the road network.
2. Store the hospital locations in a set.
3. Initialize all distances to infinity and all parent entries to -1.
4. Set the source distance to 0 and insert the source into a min-priority queue.
5. Extract the vertex with the smallest tentative distance.
6. If the extracted entry is outdated, discard it.
7. If the vertex is a hospital, reconstruct and return the shortest path.
8. Otherwise, relax all outgoing edges and update distances when shorter paths are found.

9. Repeat until a hospital is reached or the priority queue becomes empty.
10. If the queue becomes empty without reaching a hospital, report that no hospital is reachable.
11. When a road weight changes, update the corresponding edge and repeat the algorithm.

6. Pseudocode

6.1 Main Program

```
BEGIN

    INPUT V
    CREATE an adjacency list for V vertices

    INPUT E

    FOR i = 1 TO E DO
        INPUT u, v, w
        ADD edge (u, v, w)
        ADD edge (v, u, w)
    END FOR

    INPUT numberOfHospitals

    FOR i = 1 TO numberOfHospitals DO
        INPUT hospitalNode
        ADD hospitalNode to the hospital set
    END FOR

    INPUT ambulanceLocation

    (distance, path) =
        shortestPathToNearestHospital(ambulanceLocation)

    IF distance != -1 THEN
        PRINT "Initial shortest travel time:", distance
        PRINT "Path:", path
    ELSE
        PRINT "No hospital reachable"
    END IF

    INPUT u, v, newWeight
```

```
    UPDATE the weight of edge (u, v)
    UPDATE the weight of edge (v, u)

    (newDistance, newPath) =
        shortestPathToNearestHospital(ambulanceLocation)

    IF newDistance != -1 THEN
        PRINT "Updated shortest travel time:", newDistance
        PRINT "Updated path:", newPath
    ELSE
        PRINT "No hospital reachable after update"
    END IF

END
```

6.2 Function to Find the Nearest Hospital

```
FUNCTION shortestPathToNearestHospital(source)

    FOR each vertex i DO
        dist[i] = INFINITY
        parent[i] = -1
    END FOR

    dist[source] = 0

    CREATE a min-priority queue pq
    INSERT (0, source) into pq

    WHILE pq is not empty DO

        (curDist, u) = EXTRACT_MIN(pq)

        IF curDist > dist[u] THEN
            CONTINUE
        END IF

        IF u belongs to the hospital set THEN
            path = empty list
            v = u

            WHILE v != -1 DO
```

```
                APPEND v to path
                v = parent[v]
            END WHILE

            REVERSE path

            RETURN (dist[u], path)
        END IF

        FOR each neighbor v of u with edge weight w DO

            newDist = dist[u] + w

            IF newDist < dist[v] THEN
                dist[v] = newDist
                parent[v] = u
                INSERT (dist[v], v) into pq
            END IF

        END FOR

    END WHILE

    RETURN (-1, empty list)

END FUNCTION
```

6.3 Function to Update Road Weight

```
FUNCTION updateEdgeWeight(u, v, newWeight)

    FIND edge (u, v) in the adjacency list of u
    UPDATE its weight to newWeight

    FIND edge (v, u) in the adjacency list of v
    UPDATE its weight to newWeight

END FUNCTION
```

Note: The road network is assumed to be undirected, and all travel-time weights must be non-negative.

7. Time Complexity Analysis

The time complexity of Dijkstra's Algorithm depends on the graph representation and the data structure used to select the minimum-distance vertex.

Initialization:

Initializing the distance and parent arrays requires O(V) time.

Priority Queue Operations:

Each vertex can be extracted from the priority queue in logarithmic time, and each successful relaxation can insert another entry into the queue.

Using an adjacency list and a binary min-heap, the overall time complexity is:

O((V + E) log V)

Where:

- V = Number of vertices.
- E = Number of edges.

Total Time Complexity: O((V + E) log V)

For a road network with non-negative travel-time weights, this implementation efficiently calculates shortest paths. Since the algorithm is executed again after a road-weight update, the same worst-case complexity applies to each recomputation.

8. Space Complexity Analysis

The space requirements are as follows:

Component	Space Complexity
Adjacency List	O(V + E)
Distance Array	O(V)
Parent Array	O(V)
Hospital Set	O(H), where H is the number of hospitals

Priority Queue	O(V + E) in the worst case with duplicate entries

The adjacency list requires O(V + E) space. The distance array, parent array, and hospital set require O(V) additional space because H ≤ V.

The priority queue can contain duplicate entries when a better distance is discovered. Therefore, its worst-case space usage can be O(V + E).

Total Space Complexity: O(V + E)

---

9. Sample Input and Output

Sample Input

Enter number of vertices (intersections): 6
Enter number of edges (roads): 9

Enter edges (source destination weight):
0 1 10
0 2 5
1 2 2
1 3 1
2 3 9
2 4 2
3 4 4
3 5 7
4 5 6

Enter number of hospitals: 1
Enter hospital locations: 5

Enter ambulance starting location: 0

Initial Shortest Route

The initial shortest route from vertex 0 to hospital vertex 5 is:

Path: 0 → 2 → 4 → 5

Travel Time: 5 + 2 + 6 = 13 units

Initial shortest travel time: 13
Path: 0 2 4 5

Traffic Update

Suppose congestion increases the travel time of road (2, 4) from 2 units to 20 units.

Enter edge to update (u v newWeight): 2 4 20

After updating the road weight, the algorithm recalculates the shortest route.

The new shortest route is:

Path: 0 → 2 → 1 → 3 → 4 → 5

Travel Time: 5 + 2 + 1 + 4 + 6 = 18 units

After update shortest travel time: 18
Path: 0 2 1 3 4 5

Result: The system identifies the updated shortest route after the travel time of a road changes.

10. Applications

Dijkstra's Algorithm has several practical applications:

1. **Emergency Services:** Finding the shortest travel-time route for ambulances and fire engines.
2. **Navigation Systems:** Calculating routes between locations in road networks.
3. **Traffic Management:** Recalculating routes when road travel times change.
4. **Network Routing:** Finding minimum-cost paths between network nodes.
5. **Logistics and Transportation:** Optimizing delivery routes based on weighted travel costs.

11. Conclusion

Dijkstra's Algorithm provides an effective solution for finding the shortest travel-time route from an ambulance's current location to the nearest reachable hospital in a weighted road network.

By representing intersections as vertices and roads as weighted edges, the system determines the optimal route using a greedy strategy. When traffic conditions change, the affected edge weight can be updated and the shortest route recalculated.

The algorithm has a time complexity of $O((V + E) \log V)$ when implemented with an adjacency list and a binary min-heap, making it suitable for many road-network applications.

Therefore, Dijkstra's Algorithm can contribute to intelligent traffic management by supporting efficient route planning, reducing avoidable travel delays, and improving emergency response. Its effectiveness in a real smart city depends on reliable traffic data and integration with other emergency-service requirements.
